INTRODUCTION

PROJECT NAME:CONTACT MANAGER
TEAM ID :SWTID-2026-6470
TEAM SIZE :4
TEAM LEADER :SUDHA M
TEAM MEMBER:MONIKA M
TEAM MEMBER :MONISHA V
TEAM MEMBER :MONISHA J

Project overview

A Contact Manager System is an application designed to help users efficiently store and manage contact details in a digital format. Instead of writing contacts in notebooks, the system allows users to maintain their contacts electronically, making it easier to add, update, search, and delete information.
The main objective of the project is to create a user-friendly system that organizes personal or professional contacts in one place and allows quick access whenever needed.

Project Objectives

1. To create a system for storing contact information such as name, phone number, email, and address in an organized way.
2.To allow users to add new contacts easily into the system.
3.To provide a search function so users can quickly find a contact by name or phone number.
4.To enable updating of contact details whenever changes are required.
5.To allow deletion of unwanted contacts from the contact list.
6.To maintain contacts digitally instead of using manual records like notebooks.
7.To demonstrate basic database operations such as Create, Read, Update, and Delete (CRUD).
8.To develop a simple and user-friendly interface for managing contacts effectively

Target users

1. Individual user:People who want to store and manage their personal contacts such as friends, family members, and colleagues in one place.
2.Students:Students who need to keep contact information of classmates, teachers, or project team members.
3.Small Business Owners:Small businesses that need to maintain customer or client contact details for communication.
4.Office Staff:Employees who manage professional contacts like clients, suppliers, or coworkers.
5.Organizations:Organizations that need a simple system to maintain contact information of members or employees.

Pre-requisite & knowledge requirements

Developing or implementing a Contact Manager system—whether it’s a simple CLI tool or a full-scale CRM (Customer Relationship Management) platform—requires a specific blend of technical and architectural foundations.

1. Programming Fundamentals

Before diving into contact-specific logic, you need a firm grasp of a general-purpose language (like Python, Java, or JavaScript).
Data Structures: You must understand how to store contacts in memory efficiently.
Lists/Arrays: For ordered sequences.
Dictionaries/Hash Maps: Crucial for "lookups" (e.g., finding a phone number by a contact's name).
Object-Oriented Programming (OOP): This is the industry standard for contact managers. You’ll likely create a Contact class with attributes like name, email, and phone_number.
CRUD Operations: This is the "bread and butter" of the topic. You need to know how to Create, Read, Update, and Delete data entries.

2. Data Persistence & Storage

A contact manager is useless if the data vanishes when you close the program.
File I/O: At a basic level, you should know how to read from and write to .txt or .csv files.
Serialization: Knowledge of formats like JSON or XML is vital for saving complex contact objects.Databases (SQL): For professional-grade apps, you'll need to
understand relational databases like SQLite or PostgreSQL. This involves learning how to structure tables and write queries.
3. User Interface (UI) Design
How will the user interact with the contacts?
CLI (Command Line Interface): Good for beginners. Requires knowledge of input/output streams and menu loops.
GUI (Graphical User Interface): Requires libraries like Tkinter (Python), Swing (Java), or React (Web). You'll need to understand "Event-Driven Programming"—where the code waits for a user to click a "Save" button.
4. Input Validation & Security
Contacts often contain sensitive or specific data formats.
Regex (Regular Expressions): Essential for validating that an email address actually looks like an email or that a phone number has the correct amount of digits.
Error Handling: Using try/except blocks to ensure the program doesn't crash if a user enters a letter where a number should be.
Data Privacy: Basic awareness of how to handle personal information (PII) securely.
modern Contact Manager is usually built using a Tiered Architecture. Here is the breakdown of the architectural layers and the recommended tech stacks.
1. Architectural Patterns
Most Contact Managers follow one of these two structural patterns:
MVC (Model-View-Controller)
This separates the data logic from the user interface.
Model: Defines the data structure (e.g., Name, Phone, Group tags).
View: The interface the user sees (Web page or Mobile screen).

Controller: The logic that processes user input (e.g., "If user clicks delete, remove ID 402 from the Database").

N-Tier / Layered Architecture

Common for enterprise CRMs, this separates the system into:

Presentation Layer: (Frontend)

Business Logic Layer: (Service layer for validation and sorting)

Data Access Layer: (Interaction with the Database)

2. The Tech Stack Options

The "stack" you choose depends on whether you want a local tool, a web app, or a mobile contact sync tool.

A. The Modern Web Stack (Full-Stack)

This is the most common approach for accessible, cloud-synced contact manager

Frontend: React.js or Vue.js (for a responsive, fast-loading list).

Backend: Node.js (Express) or Python (FastAPI/Django).

Database: * PostgreSQL: Best for relational data (linking contacts to "Companies" or "Groups").

MongoDB: If you want a flexible schema (where some contacts have 5 emails and others have none).

Auth: Firebase Auth or Auth0 (to keep contact lists private to specific users).

B. The Mobile Stack

If you want the app to live on a phone and access the device's native address book.

Cross-Platform: Flutter or React Native.

Local Storage: SQLite or Realm (for offline access).

API: REST or GraphQL (to sync local contacts with a cloud backup).

3. Scalability & Performance Needs

If your contact manager grows to thousands of entries, your architecture needs:

Indexing: Setting up database indexes on "Last Name" so searches aren't sluggish.

Pagination: Loading only 50 contacts at a time instead of 5,000.

Caching: Using Redis to store "Frequent Contacts" for instant

Database architecture and stack

1. The Relational Schema (ERD)
A robust Contact Manager doesn't store everything in one giant table . Instead, it uses Normalization to reduce redundancy.

Users Table: Stores account info (username, hashed password) for the person owning the contact list.
Contacts Table: Stores the primary identity (First Name, Last Name, Birthday, Profile Picture URL).
Contact_Details Table: A separate table for "multi-valued" attributes like Phone Numbers or Emails. This allows one contact to have an infinite number of entries without adding columns to the main table.
Groups/Labels Table: Defines categories (e.g., "Vendors," "Family").
Contact_Groups (Junction Table): Since a contact can belong to multiple groups, we use a "Many-to-Many" relationship here.

2. Key Architectural Concepts
Normalization vs. Denormalization
Normalization: Splitting data into many tables (as shown above). This ensures that if a contact changes their last name, you only update it in one place.
Denormalization: Sometimes used in high-read environments (like a search bar) where you combine data into a single "Search Index" to make lookups lightning-fast.
Indexing for PerformanceWithout indexes, a database has to scan every single row to find
"John Doe."
Primary Keys: Unique IDs (UUIDs or Integers) for every contact.
Secondary Indexes: You should index columns like last_name, email, and user_id to ensure that searching through 10,000+ contacts takes milliseconds rather than seconds.

3.Sample SQL Architecture (PostgreSQL)

For a modern Contact Manager, your schema might look like this:

```
-- Main Contact Table
CREATE TABLE contacts (
 contact_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id UUID REFERENCES users(id),
 first_name VARCHAR(50) NOT NULL,
 last_name VARCHAR(50),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Phone Numbers (One-to-Many relationship)
CREATE TABLE phone_numbers (
 phone_id SERIAL PRIMARY KEY,contact_id UUID REFERENCES
contacts(contact_id) ON
DELETE CASCADE,
 label VARCHAR(20), -- e.g., 'Work', 'Mobile'
 number VARCHAR(20) NOT NULL
);
```

Features & api endpoints

1. Core Features

A professional contact manager is more than a digital Rolodex; it requires utility features that handle real-world data messiness.

Essential Features

Search & Filter: Fuzzy search (finding "Jon" when searching "John") and filtering by groups or last-added.

Bulk Actions: Selecting multiple contacts to delete, export, or add to a specific group.

Duplicate Detection: Logic to flag contacts with the same email or phone number to prevent clutter.

Import/Export: Support for .vcf (vCard) and .csv files to allow users to move data between Google Contacts or Outlook.

Activity Logs: A "Last Contacted" timestamp that updates when you click their email or phone link.

Advanced Features

Image Hosting: Allowing users to upload profile pictures (requires integration with storage like AWS S3).

Reminders: Integration with a calendar to notify the user of a contact's birthday.

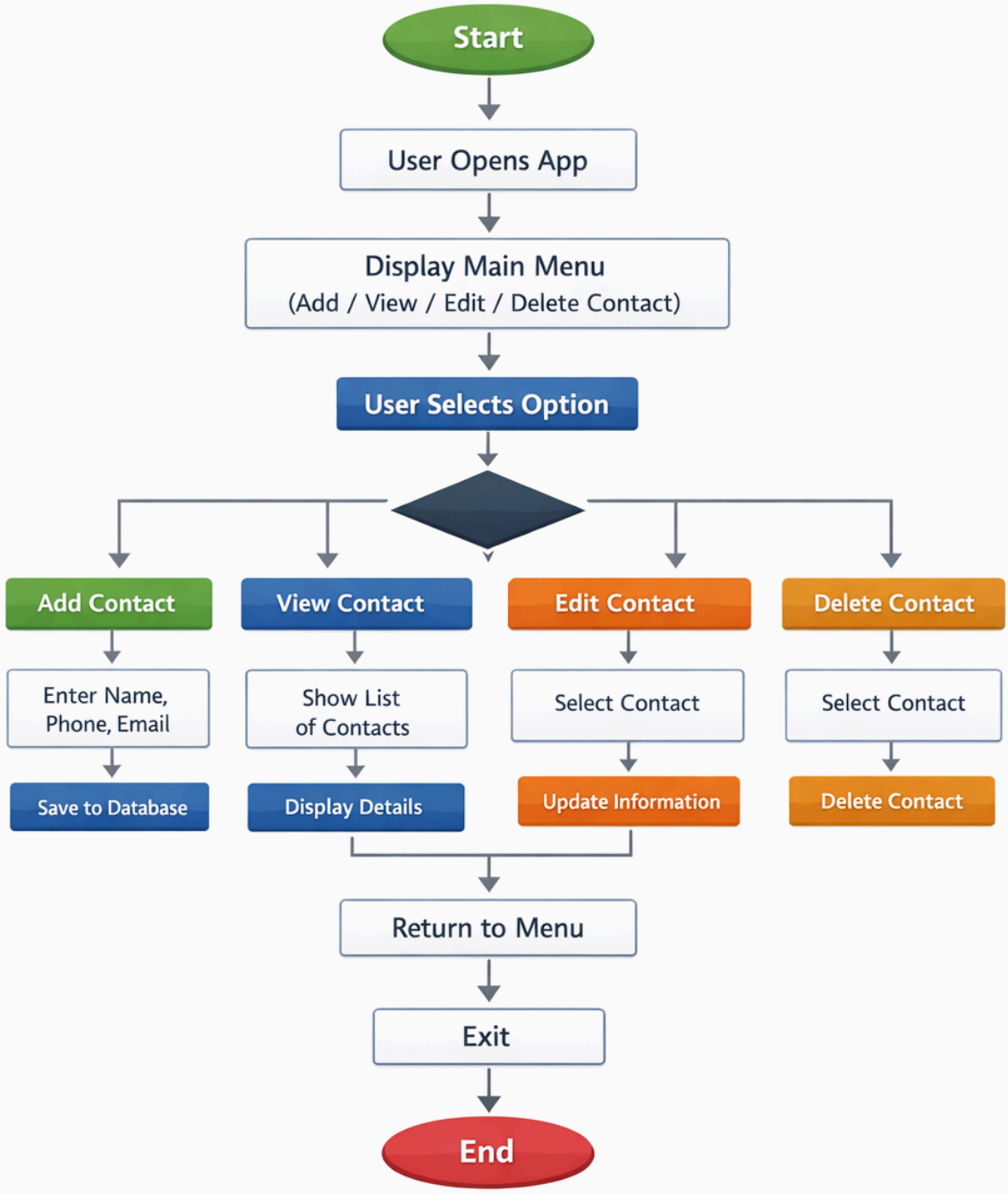
2. API Endpoints (The "How")

The API serves as the waiter between the frontend and the database.

Following REST conventions makes your API predictable and easy to integrate.

Project flowchart

Project Flowchart for Contact Manager



Project setup and configuration

Key Functional Modules

A. Authentication & Session Management

Before a user reaches the Dashboard, your configuration must include:

Session Check: Middleware to verify JWT tokens or session cookies.

Redirect Logic: Guarded routes that send unauthenticated users back to the Login Page.

B. Core CRUD Operations

Add Contact: Requires Client-side validation (e.g., Yup or Zod) before making a POST /contacts request.

Search: Implement a Debounce Timer (usually 300-500ms) to prevent overloading the database with a query for every single keystroke.

View/Edit: Uses GET /contacts/{id} for fetching and PUT /contacts/{id} for updates.

C. Advanced Services

Data Sync (Offline Capabilities): This is the most complex part of your chart. It requires comparing Local vs. Server Timestamps to handle conflicts (Keep Local, Keep Server, or Merge).

Import/Export: Logic to parse and generate CSV or vCard files,mapped directly to your database schema.